AI SMART TUTOR — SYSTEM ARCHITECTURE

Comprehensive Technical Design & Architectural Blueprint for Full-Stack Laboratory

Institution: PCET's Pimpri Chinchwad College of Engineering (PCCOE), Pune	**Class:** Third Year B.Tech (Computer Engineering)
Course: Data Exploration & Visualization Lab (DE&VL;)	**Academic Year:** 2026–2027
Stack: FastAPI • React 19 • PostgreSQL/SQLite • Scikit-Learn • Gemini	**Document Version:** 1.0 (Production Architecture)

Architectural Directive: The AI Smart Tutor is engineered as an enterprise-grade, reproducible, and pedagogically grounded full-stack system. It replaces fragmented notebook environments with a strictly verified three-tier architecture. All computations—including descriptive statistics, outlier detection, data imputation, server-side visual generation, and scikit-learn ML training—occur strictly within authorized Python services. The embedded AI Tutor is strictly grounded in verified runtime facts, preventing mathematical hallucination.

1. Core Architectural Principles

The architecture is designed around four non-negotiable engineering principles:

1. **Server-Side Computation Exclusivity:** The frontend is strictly an interactive view and command dispatcher. No data cleaning, statistical evaluation, or model training is permitted in browser JavaScript. This ensures reproducibility, security, and consistent execution.
2. **Zero Arbitrary Code Execution:** To guarantee complete server safety, students do not submit unvetted Python code strings for execution. All operations are executed through hardened, parameterized Python service functions using verified libraries.
3. **Pedagogical Fact-Grounding:** The AI Tutor is prohibited from hallucinating dataset values or evaluation scores. The backend computes verified statistical summaries and model records first, injecting them directly into the LLM system prompt.
4. **Leakage-Free ML Design:** Supervised machine learning workflows enforce strict train/test splitting prior to fitting scalers, encoders, or imputers via scikit-learn Pipelines and ColumnTransformers.

2. Three-Tier System Topology

The system decouples presentation, application business logic, and persistent storage into three distinct physical and logical layers:

Tier	Technologies	Core Responsibilities & Components
Tier 1: Presentation	• React 19 SPA • TypeScript 5.4 • Vite 5 • Tailwind CSS v4 • Lucide Icons	• Interactive Student Workbench & Sidebar Navigation • TanStack Query state caching & optimistic UI • Client-side validation & unified ApiClient interceptor • Responsive chart viewers, tables & chat interface
Tier 2: Application & Services	• FastAPI 0.110+ • Python 3.12 LTS • Pydantic v2 • Uvicorn ASGI • Argon2-cffi • Gemini SDK	• REST API v1 routing & dependency injection • Argon2id authentication & sliding session rotation • Pandas/NumPy EDA & IQR anomaly detection engine • Matplotlib/Seaborn headless chart rendering • Scikit-learn Pipeline model training & metrics • Grounded Socratic AI Tutor service
Tier 3: Persistence & Storage	• PostgreSQL 16 • SQLite fallback • SQLAlchemy 2.0 • Alembic • Local Secure FS	• Relational storage: Users, Sessions, Datasets, Runs, Models, Conversations, Progress • Private file store: /storage/uploads (raw files) • Artifact store: /storage/artifacts (PNG charts) • Report store: /storage/reports (PDF/MD downloads)

3. Frontend Presentation Tier Architecture

The frontend is structured as a modern Single Page Application (SPA) maximizing student engagement through responsive layouts, instant feedback, and accessible UI controls.

3.1 Component Hierarchy & Route Mapping

The application leverages React Router v7 with declarative route protection. Unauthenticated requests are redirected to the Login/Registration portal, while authenticated sessions are wrapped inside the persistent DashboardLayout shell.

Page / View	Route URI	Interactive Features & Capabilities
Landing Page	/	Public portal, feature showcases, curriculum alignment, CTA links.
Auth Pages	/login, /register	Argon2id credential forms, clean validation errors, session bootstrapping.
Dashboard	/dashboard	Metrics overview, owned datasets, recent runs, models, quick actions.
Dataset Explorer	/explorer	CSV/JSON upload, schema preview, pagination, null/duplicate profiling.
Cleaning Lab	/cleaning	Configurable mean/median/mode imputation, deduplication, IQR outlier audit.
Visualization Lab	/visualization	Server-rendered Matplotlib/Seaborn plots, parameter selectors, image viewer.
Statistical Lab	/statistics	Hypothesis testing, Pearson correlations, ANOVA, p-value educational notes.
ML Modeling Lab	/ml-lab	Target/feature selection, classification/regression, confusion matrix, metrics.
AI Tutor Portal	/tutor	3 Teaching modes (Guided, Direct, Quiz), streaming history, prompt hints.
Report Hub	/reports	Laboratory run aggregation, downloadable Markdown and PDF generation.

3.2 Unified API Client Architecture (`client.ts`)

All network interactions are centralized inside a singleton ApiClient class providing:

- **Dual Authentication Handling:** Automatically attaches Bearer tokens in the Authorization header and custom X-Session-Token headers while supporting credentials: 'include' for HttpOnly cookies.
- **Smart Serialization:** Automatically infers application/json payloads while passing raw FormData untouched for binary multipart file uploads.
- **Error Unpacking:** Unpacks FastAPI Pydantic validation errors (array of field-level errors) into user-friendly diagnostic messages.
- **Automatic Session Teardown:** Intercepts HTTP 401 Unauthorized responses to purge stale tokens and prompt clean re-authentication.

4. Backend Application & Service Architecture

The backend is developed purely in Python 3.12 utilizing FastAPI for high-concurrency asynchronous route handling and strict type enforcement via Pydantic v2 schemas.

4.1 Request Lifecycle & Middleware Pipeline

When an incoming HTTP request enters Uvicorn, it traverses a structured defense-in-depth pipeline:

1. **CORS Middleware:** Restricts cross-origin requests exclusively to whitelisted frontend development and production origins (e.g., ports 5173 and 3000).
2. **Security Headers Middleware:** Automatically injects X-Content-Type-Options: nosniff, X-Frame-Options: DENY, X-XSS-Protection: 1; mode=block, and strict referrer policies on every response.
3. **Dependency Injection Layer (`dependencies.py`):** Resolves database sessions via get_db, validates session tokens, checks expiration timestamps, and extracts the verified current_user context.
4. **Route Handlers & Validation:** Validates input payloads against Pydantic models. Rejects malformed requests before executing service code.
5. **Global Exception Handlers:** Catches internal errors without exposing database query strings or server stack traces to clients.

5. Scientific Computation & Machine Learning Engines

The system incorporates dedicated Python computational engines built on top of NumPy, Pandas, Matplotlib, Seaborn, and Scikit-Learn.

5.1 Data Cleaning & Preprocessing Engine (`cleaning_service.py`)

The cleaning service provides idempotent data transformations while preserving raw files:

- **Imputation:** Supports median and mean imputation for continuous variables and mode imputation for categorical attributes.
- **Deduplication:** Detects exact duplicate records, reporting before-and-after cardinalities.
- **IQR Anomaly Detection:** Computes first quartile (Q_1), third quartile (Q_3), and fences ($[Q_1 - 1.5\times\text{IQR}, Q_3 + 1.5\times\text{IQR}]$) to quantify and safely handle outliers.

5.2 Visualization Engine (`visualization_service.py`)

To avoid GUI thread contention on web servers, the visualization engine configures Matplotlib in headless Agg mode. Supported chart families include:

- **Univariate:** Histograms with Kernel Density Estimation (KDE), bar charts, and countplots.
- **Bivariate:** Scatterplots with trendlines, boxplots by category, and multi-line time series.
- **Multivariate:** Correlation heatmaps with annotated Pearson coefficients and bounded pairplots.

All figures are saved to isolated private storage (/storage/artifacts/) and streamed to the client via secure, authorized endpoint routes with custom caching headers.

5.3 Leak-Free Machine Learning Pipeline (`ml_service.py`)

In introductory data science education, data leakage is the most prevalent error. AI Smart Tutor systematically prevents leakage through the following pipeline architecture:

Pipeline Stage	Implementation & Safeguard Mechanism
1. Target Leakage Validation	Verifies that the student-selected target variable is strictly excluded from feature matrix X.
2. Isolated Data Splitting	Performs train_test_split (80/20) with fixed random seed and stratification prior to computing any summary statistics.
3. ColumnTransformer Assembly	• Numerical: SimpleImputer(strategy='median') -> StandardScaler() • Categorical: SimpleImputer(strategy='most_frequent') -> OneHotEncoder(handle_unknown='ignore')
4. Unified Model Fitting	Assembles composite Pipeline([('preprocessor', preprocessor), ('model', estimator)]). Fits strictly on $(X_{\text{train}}, Y_{\text{train}})$.
5. Metric Extraction	Evaluates on X_{test}: Classification (Accuracy, Precision, Recall, F1, Confusion Matrix); Regression (MAE, RMSE, R^2).

6. Generative AI Tutor & Grounded Pedagogical Engine

The AI Tutor is built upon an isolated LLM Provider Gateway supporting Google Gemini (with model fallbacks) and an educational deterministic mock engine. The tutor architecture resolves common LLM deficiencies:

6.1 Grounded Context Injection Pipeline

Before dispatching any user prompt to the LLM, the backend queries the database for the student's active dataset metadata, calculated descriptive statistics, and model parameters. This context payload is serialized into JSON and injected into the system prompt with strict negative constraints:

```
System Prompt Directive:
"You are the AI Smart Tutor for PCCOE Computer Engineering students. You are strictly conditioned on the verified dataset facts provided below. Do NOT invent or extrapolate statistics, correlation coefficients, or model metrics. If the user asks for a calculation that is not present in the verified context, explicitly direct them to run that specific analysis in the laboratory workbench."
```

6.2 Pedagogical Modes of Operation

- **Guided Learning Mode:** Employs the Socratic method. Instead of spitting out direct solutions, the tutor asks thought-provoking questions, pointing out skewness or distribution shape.
- **Direct Explanation Mode:** Provides rigorous, syntax-accurate explanations of NumPy/Pandas operations, Matplotlib layout parameters, or Scikit-learn algorithms.
- **Quiz & Viva Mode:** Generates multiple-choice and conceptual viva examination questions derived directly from the student's dataset metrics and trained model results.

7. Database Design & Entity Relationship Architecture

The persistence layer utilizes SQLAlchemy 2.0 ORM with asynchronous capabilities, targeting PostgreSQL 16 in production with seamless SQLite local testing compatibility. All entities utilize tamper-proof UUID primary keys.

Entity Table	Key Attributes & Datatypes	Constraints & Relationships
users	id (UUID), email (String, unique), password_hash (String), role (Enum), is_active (Bool)	Primary user registry. Enforces email uniqueness and Argon2id hash storage.
sessions	id (UUID), user_id (FK), token_hash (String), expires_at (DateTime), revoked_at	Stateful session tracking. Supports sliding expiry and instant logout revocation.
datasets	id (UUID), owner_id (FK), name (String), storage_key, file_format, row_count, col_metadata (JSON)	Tracks uploaded files, row/column cardinality, and cached schema profile.
analysis_runs	id (UUID), dataset_id (FK), run_type (Enum), parameters (JSON), results (JSON), artifact_path	Historical ledger of all cleaning operations, statistical tests, and chart runs.
ml_models	id (UUID), dataset_id (FK), model_type, target_col, features (JSON), metrics (JSON), artifact_key	Stores serialized trained pipelines, evaluation scores, and confusion matrix.
tutor_conversations	id (UUID), user_id (FK), dataset_id (FK), title, messages (JSON), mode (Enum)	Persists full conversation turns, system prompts, and student questions.
student_progress	id (UUID), user_id (FK), completed_topics (JSON), quiz_scores (JSON), last_activity	Audits student laboratory milestones and viva evaluation scores.

8. Application Security & Defense-in-Depth

Security is woven directly into every layer of the system to prevent common academic software vulnerabilities:

- **Argon2id Password Hashing:** Passwords are hashed using Argon2id with 64 MB memory cost, 3 iterations, and 4 parallel threads, providing state-of-the-art resistance to GPU/ASIC cracking.
- **Cryptographic Session Hashing:** Session tokens are generated using secrets.token_urlsafe(32) and stored in the database exclusively as SHA-256 hashes, mitigating token extraction in database breaches.
- **File Storage Quarantine:** Uploaded files are assigned random UUID storage keys and stored outside public web directories. File extension and MIME validation prevent executable upload attacks.
- **Rate Limiting Safeguards:** Strict rate limits are enforced across authentication (20 req/min), dataset upload (15 req/min), and AI tutor queries (30 req/min).

9. Deployment Topology & DevOps Infrastructure

The system provides complete multi-container orchestration via Docker Compose with automated service dependencies and health probes:

- **Postgres Container:** Official postgres:16-alpine with healthcheck pg_isready on port 5432.
- **Backend Container:** Lightweight Python 3.12-slim executing Alembic migrations on startup followed by Uvicorn on port 8000.
- **Frontend Container:** Multi-stage build (Node 20 Alpine builder -> Nginx Alpine server) serving optimized static assets and reverse-proxying /api/ to the backend.
- **Automated Verification:** The test suite (14 passing automated tests) validates authentication, file ingestion, statistical calculations, ML pipelines, and tutor responses prior to deployment.

Prepared by:	**Approved by:**
Third Year B.Tech Computer Engineering Students	Department of Computer Engineering, PCCOE